

CAPUTO FOODS · QUALITY CONTROL

SAFETY

Mapa del sistema

Cómo está hecho, qué hace cada parte y por dónde se conecta todo

50

ARCHIVOS JS

773

FUNCIONES

13.694

LÍNEAS

30

PANTALLAS

Documento de trabajo para depurar. Cada dato sale de leer el código, no de memoria: el inventario se sacó con un script sobre el repositorio.

Versión de la aplicación `safety-qc-v64` · repositorio `kamilo16jc/SAFETY` · 28 de septiembre de 2026

Contenido

- 1 El sistema en una página
- 2 Las cuatro capas y sus archivos
- 3 Los datos: dónde vive cada cosa
- 4 El viaje de un dato hasta Firebase
- 5 Quién puede ver qué
- 6 Los seis módulos, uno por uno
- 7 Flujo: producción → laboratorio → COA
- 8 Flujo: inspección → hold → CAPA
- 9 Flujo: materia prima → grilling cheese
- 10 Las costuras: qué usa todo el mundo
- 11 Puntos frágiles
- 12 Por dónde empezar a depurar
- 13 Glosario de registros

1 • El sistema en una página

SAFETY es una aplicación de una sola página. No hay servidor propio: el navegador guarda todo en el equipo y lo sincroniza con Firebase cuando hay red. Eso explica casi todas las decisiones de diseño que siguen.

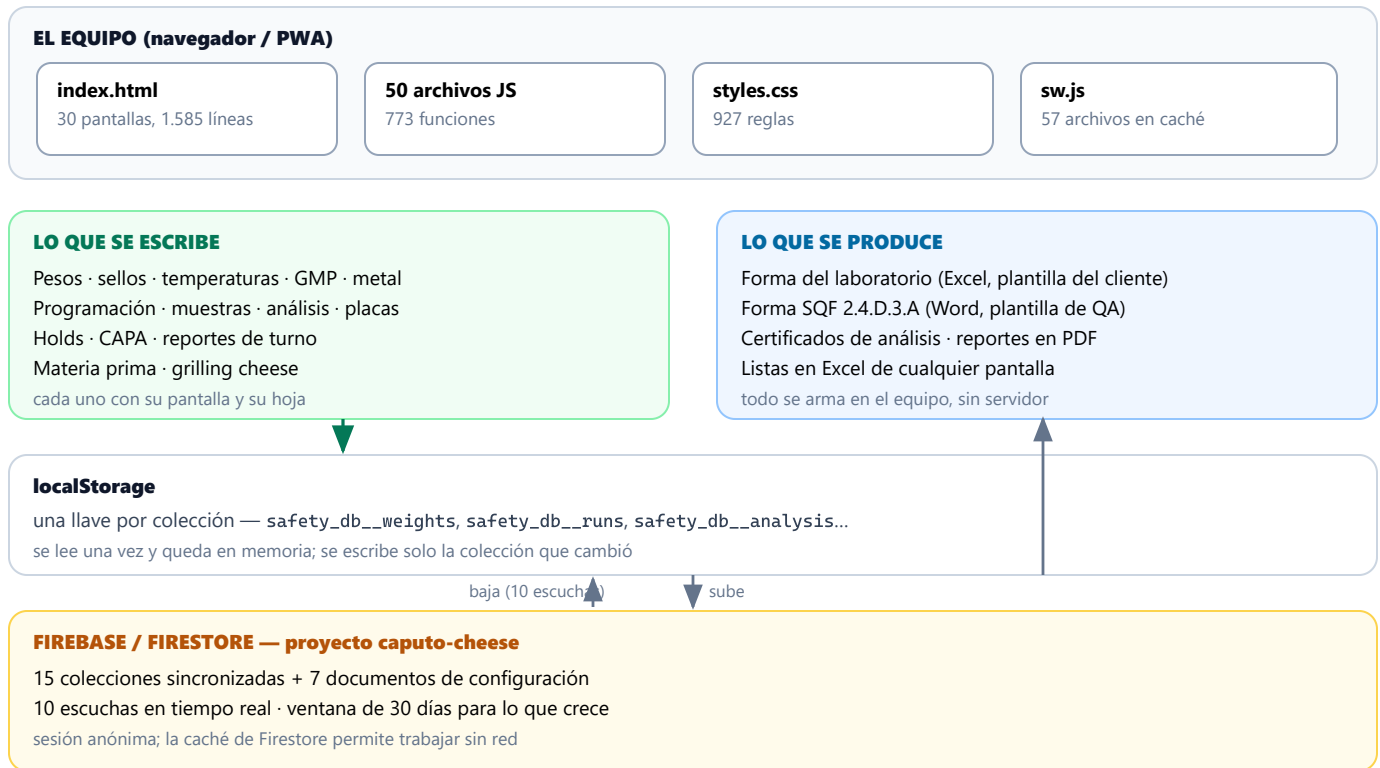


Diagrama 1 — Las piezas y por dónde pasa la información.

Lo que hay que retener: ninguna pantalla habla con Firebase directamente. Todas escriben con `saveDB()` en el equipo, y una sola capa — `js/core/firebase.js` — se encarga de subirlo. Por eso la aplicación sigue funcionando sin red y por eso un error de sincronización casi siempre está en ese archivo, no en la pantalla.

2 · Las cuatro capas y sus archivos

Los 50 archivos no están sueltos: hay cuatro capas con responsabilidades distintas. Cuanto más abajo, más gente depende de ese archivo.

EXPORT · 5 archivos — lo que sale impreso o en Excel

xlsx-write · pdf-reports · dash-pdf · shift-pdf · capa-pdf · excel
no guardan nada: leen y arman el archivo

INTEGRATIONS · 4 archivos — las plantillas de la casa

word-fill (pesos) · metal-fill · excel-fill (GMP) · grilling-fill (SQF 2.4.D.3.A)
abren el .docx/.xlsx original con JSZip y cambian los marcadores

MODULES · 27 archivos — una pantalla, un archivo

weight · seal · metal · capa · shift · holds · samplelist · production · lab · lab-form ·
lab-tests · analysis · yeast · raw · grilling · coa · products · catalog · search · reports · dashboard · admin · activity...

CORE · 11 archivos — lo que usan todos

data (la base) · firebase (sincronía) · nav (navegación) · modules (accesos) · auth · utils · icons · mobile · dup-guard · theme · sound · pwa

Diagrama 2 — Las capas. Las flechas de dependencia van siempre hacia abajo.

Los archivos más grandes

ARCHIVO	CAPA	LÍNEAS	FUNC.	QUÉ RESUELVE
modules/search.js	Módulo	1.047	50	Buscar por lote/producto y el visor-editor de registros
modules/coa.js	Módulo	765	48	Certificados, estado, resultados del laboratorio
modules/holds.js	Módulo	731	26	Retenciones: alta, liberación, seguimiento
modules/products.js	Módulo	723	54	Catálogo, clientes, tests por producto
modules/dashboard.js	Módulo	691	38	Indicadores del día y gráficas
modules/reports.js	Módulo	675	14	Reportes por rango y sus exportaciones
modules/raw.js	Módulo	621	40	Las cuatro hojas de materia prima
core/firebase.js	Core	472	28	Subir, bajar, escuchar y resolver conflictos
modules/production.js	Módulo	458	31	Programación y ciclo de vida de la muestra
modules/analysis.js	Módulo	390	29	Análisis de muestra y su hoja

Dato útil para depurar: search.js y coa.js suman 1.812 líneas y 98 funciones entre los dos. Son los dos sitios donde más cuesta encontrar un error y los primeros candidatos a partir en archivos más pequeños.

3 · Los datos: dónde vive cada cosa

Todo el sistema gira sobre un único objeto en memoria con 15 colecciones. Saber quién escribe cada una es la mitad del trabajo de depurar.

COLECCIÓN	QUÉ GUARDA	QUIÉN LA ESCRIBE	QUIÉN LA LEE	SINCRONÍA
weights	Pesos y estados de línea	weight.js	search, reports, dashboard, shift	ventana 30 d
seals	Sellado de bolsa	seal.js	search, reports, dashboard	ventana 30 d
temps	Temperatura y humedad	dashboard.js	reports, dashboard	ventana 30 d
gmps	Auditoría GMP	dashboard.js	reports, excel-fill	ventana 30 d
metal	Detector de metales	metal.js	reports, metal-fill	ventana 30 d
analysis	Análisis + placa + COA	analysis, yeast, coa	coa, search, reports	ventana 30 d
raw	Materia prima (4 secciones)	raw.js	grilling.js	ventana 30 d
runs	Programación y muestras	production, lab, lab-form	samplelist, lab, analysis, coa	completa
products	Catálogo de producto	products, catalog	casi todos	completa
holds	Retenciones	holds.js	search, dashboard	completa
capa	Incidentes y CAPA	capa.js	search, capa-pdf	completa
shifts	Reportes de turno	shift.js	search, shift-pdf	completa
grilling	Recepción, sensory, firma	grilling.js	grilling-fill	completa
suppliers	Proveedores	raw.js	raw.js	completa
rawItems	Materiales recibidos	raw.js	raw, grilling	completa

Y aparte, la configuración

Siete documentos que no son registros sino ajustes compartidos. Viven en `config/` dentro de Firestore y bajan enteros a cada equipo.

DOCUMENTO	QUÉ ES	POR QUÉ IMPORTA
customers	Clientes, sus formas y sus tests	Confidencial: nunca va al repositorio
labFormHeaders	Columnas de cada forma de laboratorio	De aquí sale dónde va cada X
labTestMap	Nombre genérico → columna de la forma	Traduce «Salmonella» a la casilla real
labCounters	Numeración de muestras por semana	Contador corrido compartido entre equipos
operators	Las personas	Lista única; el login lee de aquí
roles	Los roles y sus casillas	Define qué ve y qué puede cada uno
areas	Áreas de planta	Reportes de turno y GMP

Ojo al depurar: las colecciones con ventana de 30 días no están completas en memoria. Si un filtro pide algo más viejo, la pantalla llama a `loadHistory()` y hasta que no responde, la lista se ve vacía. Un «no aparece mi registro de hace dos meses» casi nunca es un dato perdido.

4 • El viaje de un dato hasta Firebase

Este es el camino que recorre cualquier registro, desde que alguien pulsa guardar hasta que aparece en el teléfono de otro. Entenderlo evita perseguir fantasmas.



Diagrama 3 — De la pantalla a Firestore y de vuelta.

5 • Quién puede ver qué

Desde hace poco el acceso no lo decide el rol de fábrica sino un catálogo de roles que se edita en pantalla. Este es el camino completo de una casilla marcada hasta una pantalla que no se abre.

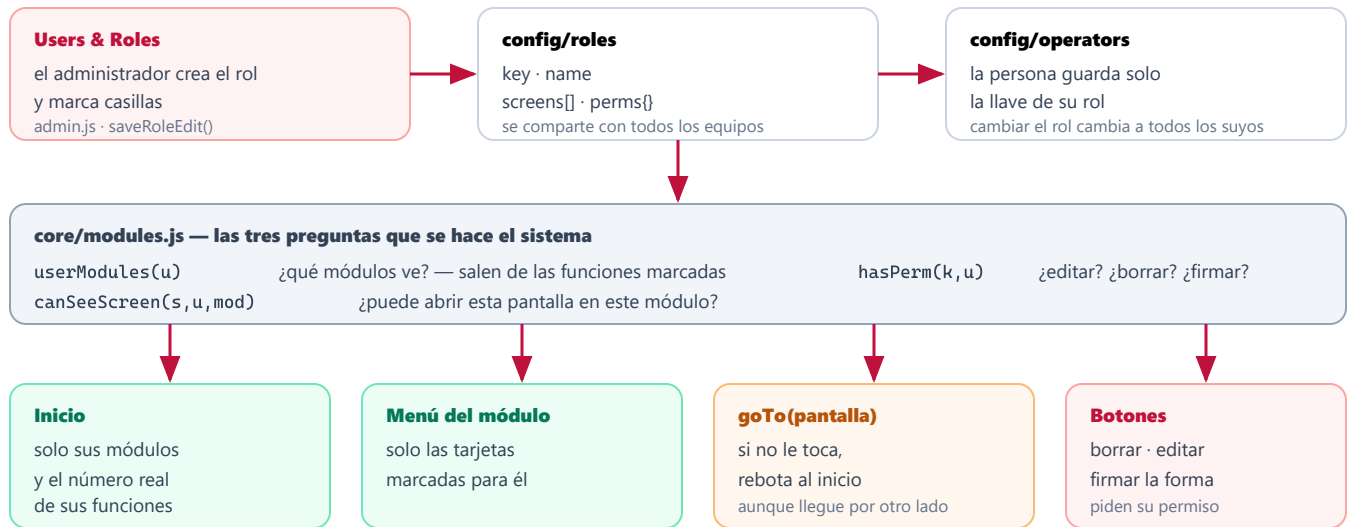


Diagrama 4 — Del catálogo de roles a la pantalla que se abre o no.

La función se guarda con su módulo delante — `qa:screen-reports` — porque Search, Reports y Shift Report aparecen en varios módulos y no es lo mismo dejar ver los de QA que los del laboratorio. Los tres candados que no se pueden abrir: no se borra el rol de administrador, no se borra un rol con gente, y nadie se deja a sí mismo fuera del panel.

6 • Los seis módulos, uno por uno

QA INSPECTION 11 funciones

PANTALLA	ARCHIVO	QUÉ HACE Y QUÉ DEJA ESCRITO
Weights	weight.js	Hasta 5 pesos por chequeo; calcula promedio y cumplimiento contra el rango del paquete. Si la línea está parada, guarda el estado en vez de pesos → <code>weights</code>
List Samples	samplelist.js	Qué muestras hay que recoger hoy y cuántas; marca la recolección → <code>runs</code>
Seal Bags	seal.js	Visual, dunk tank e impresión → <code>seals</code>
Temperatures	dashboard.js	Temperatura y humedad por área → <code>temps</code>
GMP Audit	dashboard.js	Checklist por área → <code>gmps</code>
Metal Detector	metal.js	Prueba de detector con sus testigos → <code>metal</code>
CAPA Reports	capa.js	Incidente, investigación y acción correctiva → <code>capa</code>
Hold Cases	holds.js	Retención de producto y su liberación → <code>holds</code>
Shift Report	shift.js	Reporte de turno; solo manager o administrador lo modifican → <code>shifts</code>
Reports / Search	reports.js · search.js	Consulta; no escriben nada salvo al editar un registro

QA LABORATORY 11 funciones

PANTALLA	ARCHIVO	QUÉ HACE Y QUÉ DEJA ESCRITO
Sample Analysis	analysis.js	Moisture, fat y pH del día; numera el análisis del año → <code>analysis</code>
Yeast & Mold	yeast.js	Vuelve sobre ese mismo análisis a los 5 días para leer la placa → <code>analysis</code>
Lab Samples	lab.js · lab-form.js	Muestras que van al laboratorio y la forma del cliente; al generarla quedan enviadas → <code>runs</code>
Grilling Cheese	grilling.js	SQF 2.4.D.3.A en tres manos: recepción, laboratorio, sensory → <code>grilling</code>
Raw · Dry · Formag · R&D	raw.js	Cuatro hojas con el mismo motor; placa a 5 días → <code>raw</code> , <code>suppliers</code> , <code>rawItems</code>

PRODUCTION • PRODUCTS • COA • ADMINISTRATOR

PANTALLA	ARCHIVO	QUÉ HACE Y QUÉ DEJA ESCRITO
Schedule	production.js	Qué producto va en qué línea; de aquí nace la muestra → <code>runs</code>
Product Catalog / Add	products.js · catalog.js	Producto, su cliente, su rango de peso y sus tests → <code>products</code> , <code>config/customers</code>
COA Generator	coa.js	Un certificado por cliente; revisiones R2, R3... → <code>analysis</code>
Status	coa.js	Pending QA Lab / Pending Lab / Completed, con la regla de los 5 días y el domingo
Lab Results	coa.js	Lo que devuelve el laboratorio externo → <code>analysis</code>
Dashboard	dashboard.js	Indicadores del día; no escribe
Activity Log	activity.js	Quién hizo qué; lo escriben todas las pantallas
Users & Roles	admin.js	Personas y roles → <code>config/operators</code> , <code>config/roles</code>

7 · Flujo: producción → laboratorio → COA

Es el flujo más largo del sistema y el que más módulos cruza. Un solo registro —el `run`— va cambiando de estado y de dueño.

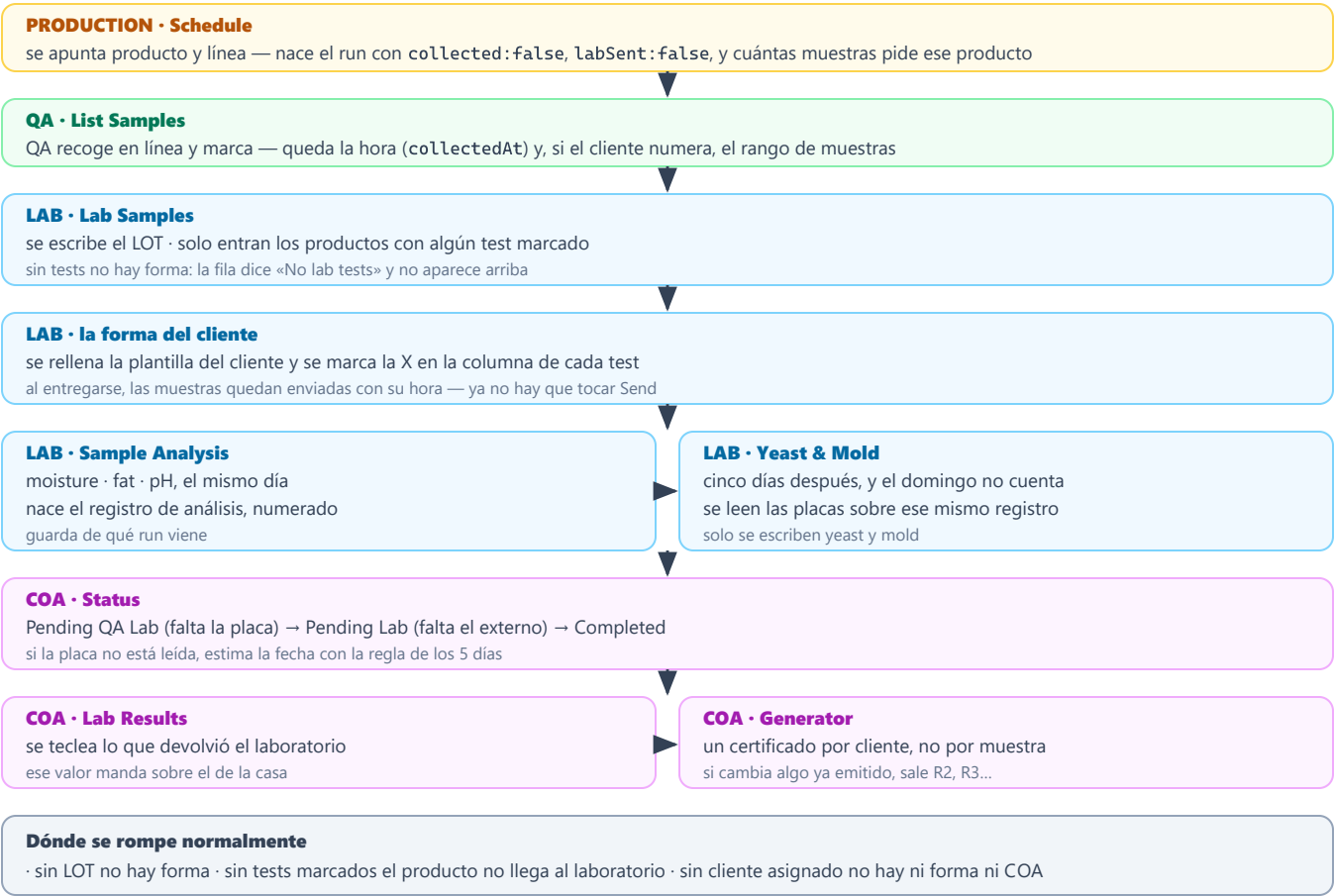


Diagrama 5 — El camino completo de una muestra, de la línea al certificado.

8 • Flujo: inspección → hold → CAPA

El otro gran circuito. Aquí el disparador no es una muestra sino un resultado que se sale de rango.

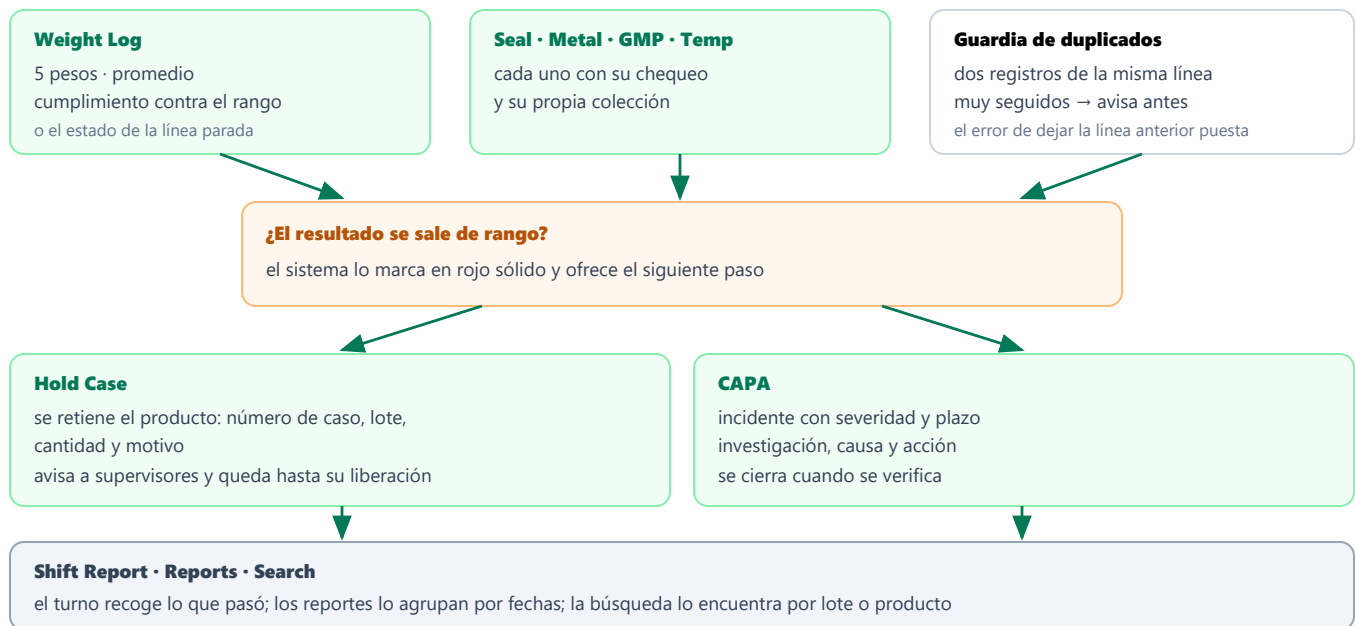


Diagrama 6 — Del chequeo que falla al documento de calidad.

Las cinco pantallas de captura comparten el mismo esqueleto: elegir línea y turno, escribir, guardar con `saveDB` y anotar en el registro de actividad. Si una falla y las otras no, el problema está en su archivo; si fallan todas, está en `core/data.js` o en `core/firebase.js`.

9 · Flujo: materia prima → grilling cheese

El circuito más nuevo. Empieza en el muelle de recepción y termina en una forma de Word firmada.

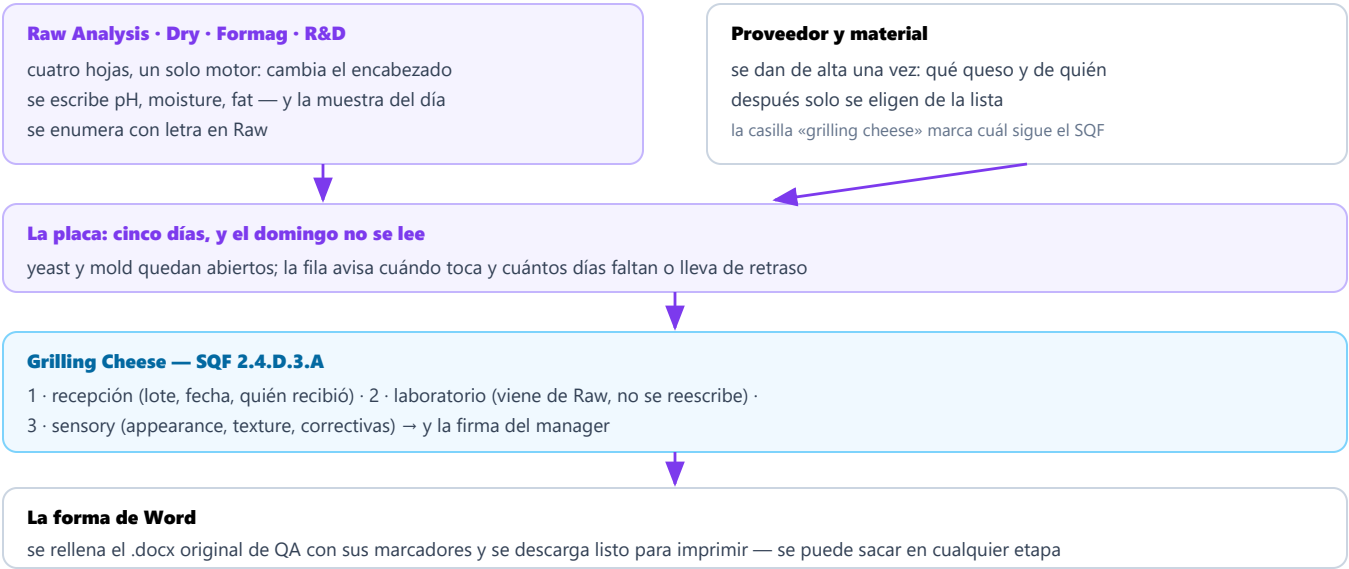


Diagrama 7 — De la recepción a la forma firmada.

El estado de un lote de grilling cheese se calcula, no se guarda: **Pending Lab** si falta pH, fat o moisture; **Pending Sensory** si falta appearance o texture; **Pending Approval** si falta la firma; **Completed** con todo. Por eso nunca se queda «pegado» en un estado viejo.

10 • Las costuras: qué usa todo el mundo

Estas son las funciones que más archivos comparten. Tocar una de ellas mueve medio sistema — y por eso son también las primeras sospechosas cuando algo falla en todas las pantallas a la vez.

FUNCIÓN	VIVE EN	USOS	ARCHIVOS	PARA QUÉ
toast	core/nav.js	195	35	El aviso de abajo: todo el mundo habla por ahí
getDB	core/data.js	105	29	La base en memoria
saveDB	core/data.js	56	20	Guardar una colección
logActivity	modules/activity.js	50	25	Quién hizo qué
localDateStr	core/utils.js	50	19	La fecha de hoy, en hora local
localISOStr	core/utils.js	31	14	La marca de tiempo de un registro
findProduct	modules/products.js	30	13	Del número al producto del catálogo
saveToFirestore	core/firebase.js	29	17	Subir un registro
echoDateRange	core/utils.js	25	10	Escribir el rango en letra, porque el calendario cambia de formato
fmtDate	modules/search.js	24	9	La fecha como se lee en pantalla
newRecordId	core/data.js	19	15	El identificador único del registro
getInitials	modules/admin.js	14	7	Las iniciales de quien está dentro
productCustomer	modules/products.js	9	4	De un producto a su cliente y sus tests
downloadCSV	modules/analysis.js	8	6	Bajar cualquier lista en Excel

Las dependencias más fuertes entre archivos

ESTE ARCHIVO...	...DEPENDEN DE	LLAMADAS	POR QUÉ
modules/catalog.js	modules/products.js	11	El catálogo es la cara del mismo dato
modules/shift.js	core/data.js	8	Turnos, áreas y rangos
modules/scanner.js	modules/products.js	7	El escáner resuelve el número a producto
modules/lab.js	modules/production.js	5	Las muestras del laboratorio son runs
modules/coa.js	modules/analysis.js	5	El certificado se arma sobre el análisis
modules/products.js	modules/lab-tests.js	5	Los tests del producto salen de la forma del cliente
export/dash-pdf.js	modules/dashboard.js	5	El PDF reusa los cálculos de la pantalla

11 • Puntos frágiles

Nada de esto está roto hoy, pero es donde el sistema se rompe cuando se rompe. Ordenado por lo que más caro sale.

1 • Funciones de núcleo que viven en un módulo

`fmtDate` está en `search.js` y la usan 9 archivos. `downloadCSV` está en `analysis.js` y la usan 6. `getInitials` está en `admin.js` y la usan 7. `findProduct` y `productCustomer` están en `products.js` y las usan 13 y 4. Mientras `index.html` los cargue todos no pasa nada, pero cualquier cambio de orden o una carga parcial deja media aplicación sin fecha. **Arreglo:** moverlas a `core/utils.js`; es mecánico y de bajo riesgo.

2 • La caché del service worker

Si se publica un cambio sin subir `CACHE = 'safety-qc-vN'`, los equipos siguen con el JavaScript viejo y el error «ya arreglado» reaparece. Nos pasó hoy mismo en el navegador de pruebas. **Arreglo:** es disciplina, pero se puede automatizar poniendo la versión en el momento de publicar.

3 • Cualquier registro sin `_fbId` se sube

La cola de reintento no distingue un dato real de uno de prueba: si está en el equipo y no tiene `_fbId`, acaba en Firestore. Por eso las pruebas se hacen con la subida interceptada. **Arreglo:** una marca explícita de «esto es de prueba» que la cola respete.

4 • Cada módulo calcula su propio «pendiente»

Analysis, Raw, Grilling, Lab y COA tienen cada uno su función de estado, parecidas pero no iguales. Son cinco sitios que hay que tocar cuando cambia una regla —como pasó con la de los cinco días y el domingo—. **Arreglo:** una sola función de estado con la regla dentro.

5 • Diez renderizadores de hoja casi iguales

Cada pantalla nueva copia el mismo patrón de `.sheet`: cabecera, fila de alta, filas, botón de guardar. Funciona, pero un arreglo de estilo hay que repetirlo diez veces. **Arreglo:** una función que arme la hoja a partir de la lista de columnas; `raw.js` ya lo hace para sus cuatro secciones y sirve de modelo.

6 • `index.html` con las 30 pantallas juntas

1.585 líneas en un archivo. Es incómodo de leer y fácil de romper. **Arreglo:** mover el marcado de cada pantalla a su módulo, como ya hacen Raw y el panel de accesos, que se pintan desde JavaScript.

7 • La ventana de 30 días no se anuncia

Cuando se busca algo más viejo, la pantalla pide el histórico y mientras tanto se ve vacía. **Arreglo:** un aviso de «buscando en el histórico» mientras llega.

12 · Por dónde empezar a depurar

Un orden que va de lo que más duele a lo que menos, y que deja el sistema más fácil de arreglar después.

#	QUÉ	POR QUÉ PRIMERO	RIESGO
1	Mover las funciones compartidas a <code>core/utils.js</code>	Quita la dependencia invisible entre pantallas que no tienen nada que ver	Bajo
2	Una sola función de estado (pendiente / completo)	Hoy una regla se cambia en cinco sitios y se olvida uno	Bajo
3	Marca de «dato de prueba» que la cola respete	Protege la base real de las pruebas	Medio
4	Partir <code>search.js</code> y <code>coa.js</code>	Son 1.812 líneas: ahí es donde cuesta encontrar los errores	Medio
5	Un armador de hojas común	Diez copias del mismo patrón	Medio
6	Sacar las pantallas de <code>index.html</code>	Deja el archivo manejable	Alto

Cómo leer un error cuando aparezca

LO QUE SE VE	DÓNDE MIRAR PRIMERO
Falla una sola pantalla	Su archivo en <code>js/modules/</code>
Fallan todas las pantallas de captura	<code>core/data.js</code>
Se guarda pero no le llega a los demás	<code>core/firebase.js</code> — cola y escuchas
Aparece un registro repetido	El identificador del documento y <code>_fbId</code>
No aparece un registro viejo	La ventana de 30 días · <code>loadHistory</code>
Alguien ve algo que no debería	<code>core/modules.js</code> · el rol y sus casillas
La forma sale mal rellena	<code>js/integrations/</code> y la plantilla
El arreglo «no llegó» al equipo	La versión de la caché en <code>sw.js</code>

13 · Glosario de registros

Los campos que de verdad se usan en cada tipo de registro. Útil cuando hay que mirar la base por dentro.

run — programación y muestra

id · date · shift · line · product · productName · labSamples · labSample · lot · collected · collectedAt · sampleFrom · sampleTo · labSent · labSentAt · notes · createdBy · createdAt

weight — peso o estado de línea

id · date · line · shift · pkg · pkgLabel · unit · vals[] · avg · pass · total · compliance · time · lot · product · productName · bagsPerCase · comments · initials · issue

analysis — análisis, placa y COA

id · seq · date · product · cheese · customerId · customer · lot · prodDate · order · po · moisture · fat · ph · yeast · mold · ymAt · testedBy · runId · coaNo · coaRev · coaAt · coaBy · mx...(resultados externos)

raw — materia prima

id · section (raw|dry|formag|rd) · date · sample · material · code · batch · supplier · time · prodDate · expDate · moisture · fat · ph · aw · yeast · mold · plateAt · plateBy · notes · testedBy

grilling — SQF 2.4.D.3.A

id · rawId · receivedDate · receivedBy · appearance · appearanceNote · texture · textureNote · corrective · sensoryDate · sensoryBy · approvedBy · approvedAt

seal — sellado

id · date · line · shift · checks{visual, dunk, printing} · time · lot · product · productName · comments · initials

hold — retención

id · caseNumber · product · lot · quantity · line · reason · initiatedBy · status (hold|released) · createdAt · history[] (cada paso con su fecha, estado, comentario y quién)

capa — incidente

id · reportNumber · capaDate · complaint · product · productName · lot · severity · status · problem · description · createdBy · createdAt

shift — reporte de turno

id · reportNumber · date · shift · area · line · category · product · lot · status · notes · action · reportedBy · supervisor · followUp

operator / role

operator: id · name · username · email · passHash · role · createdAt
role: key · name · screens[] · perms{edit, remove, approve} · protected

Cómo se hizo este documento: un script recorrió los 50 archivos del repositorio y sacó funciones, llamadas entre archivos y colecciones tocadas. Los números —773 funciones, 13.694 líneas, 195 usos de `toast` — salen de ahí, no de una lectura a ojo. Si el código cambia, se vuelve a correr y el mapa se actualiza.